

CHƯƠNG 1. TỔNG QUAN VỀ WEB, MÔI TRƯỜNG PHÁT TRIỂN VÀ QUẢN LÝ MÃ NGUỒN

Chương mở đầu thiết lập nền tảng khái niệm và công cụ cho toàn bộ học phần. Người học được trang bị hiểu biết về cách vận hành của World Wide Web, làm chủ môi trường phát triển tích hợp, tiếp cận có hệ thống về quản lý phiên bản mã nguồn, và hoàn thành chu trình khép kín từ viết mã đến công bố sản phẩm trực tuyến. Đây là những năng lực nền tảng, mang tính bắt buộc đối với mọi kỹ sư phần mềm hướng web.

1.1. Mục tiêu học tập (Learning Outcomes)

Sau khi hoàn thành chương này, sinh viên có khả năng:

Về kiến thức

- Trình bày được kiến trúc Client – Server và mô tả chi tiết chu trình Yêu cầu – Phản hồi (Request – Response) của một giao dịch web.
- Giải thích được bản chất của giao thức HTTP/HTTPS, cấu trúc URL và ý nghĩa các nhóm mã trạng thái.
- Phân biệt được website tĩnh và website động, cũng như vai trò của tầng frontend và backend.
- Phân tích được vai trò của hệ thống quản lý phiên bản và mô hình ba vùng làm việc của Git.

Về kỹ năng

- Cài đặt và cấu hình được môi trường phát triển Visual Studio Code cùng các tiện ích mở rộng thiết yếu.
- Thực hiện được quy trình quản lý mã nguồn cơ bản với Git (khởi tạo, ghi mốc, đồng bộ với kho từ xa).
- Triển khai (deploy) được một trang web tĩnh lên nền tảng GitHub Pages và thu được một địa chỉ công khai.

Về thái độ

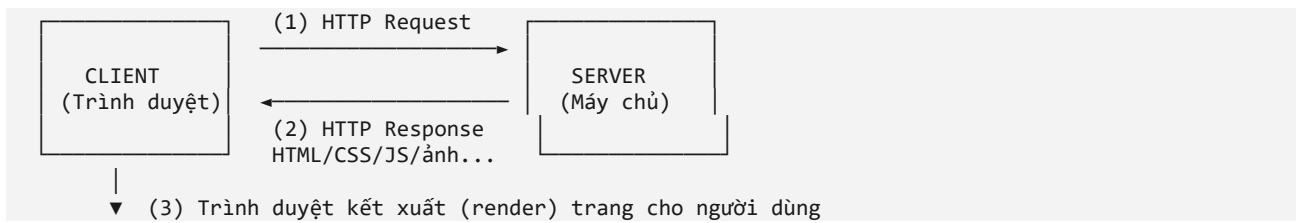
- Hình thành ý thức quản lý mã nguồn có kỷ luật, ghi mốc thay đổi rõ ràng và thường xuyên.
- Rèn luyện tinh thần trách nhiệm và tính minh bạch khi cộng tác trên một kho mã dùng chung.

1.2. Nội dung lý thuyết chuyên sâu

1.2.1. Kiến trúc Client – Server và mô hình Yêu cầu – Phản hồi

Kiến trúc Client – Server (Máy khách – Máy chủ) là mô hình tính toán phân tán nền tảng của hầu hết các ứng dụng web. Trong mô hình này, hệ thống được phân chia thành hai vai trò tách biệt: máy khách (client) là bên khởi tạo yêu cầu và trình bày kết quả — trong ngữ cảnh web chính là trình duyệt; máy chủ (server) là bên tiếp nhận, xử lý yêu cầu và trả về tài nguyên tương ứng. Sự phân tách trách nhiệm này cho phép một máy chủ phục vụ đồng thời nhiều máy khách, đồng thời tách bạch công việc phát triển giao diện (frontend) khỏi công việc xử lý nghiệp vụ (backend).

Một giao dịch web hoàn chỉnh diễn ra theo chu trình Yêu cầu – Phản hồi, được mô tả bằng sơ đồ khối dưới đây:



Giải thích: Người dùng nhập địa chỉ; trình duyệt gửi một HTTP Request đến máy chủ (1); máy chủ xử lý và trả về HTTP Response chứa các tệp cần thiết (2); cuối cùng trình duyệt phân tích và kết xuất các tệp này thành trang web hiển thị (3). Toàn bộ chu trình thường hoàn tất trong vài trăm mili-giây.

1.2.2. Giao thức HTTP/HTTPS, URL và mã trạng thái

HTTP (HyperText Transfer Protocol) là giao thức tầng ứng dụng quy định khuôn dạng và quy tắc trao đổi dữ liệu giữa client và server. Một đặc trưng cốt lõi của HTTP là tính phi trạng thái (stateless): mỗi yêu cầu được xử lý độc lập, máy chủ không mặc nhiên lưu giữ ngữ cảnh của các yêu cầu trước đó. Để duy trì trạng thái phiên làm việc (ví dụ trạng thái đăng nhập), các cơ chế hỗ trợ như cookie hoặc token được sử dụng. HTTPS (HTTP Secure) bổ sung một tầng mã hóa SSL/TLS, bảo đảm tính bảo mật và toàn vẹn của dữ liệu trên đường truyền; đây là yêu cầu bắt buộc đối với các ứng dụng có xử lý thông tin nhạy cảm.

Mỗi tài nguyên trên web được định danh bằng một URL (Uniform Resource Locator). Kèm theo mỗi phản hồi, máy chủ trả về một mã trạng thái ba chữ số, được phân loại theo chữ số đầu tiên. Bảng dưới đây tổng hợp các nhóm mã và ví dụ tiêu biểu:

Nhóm	Ý nghĩa	Ví dụ tiêu biểu
2xx	Thành công	200 OK — yêu cầu được xử lý thành công
3xx	Chuyển hướng	301 Moved Permanently — tài nguyên đã đổi địa chỉ
4xx	Lỗi phía client	404 Not Found — không tìm thấy tài nguyên
5xx	Lỗi phía server	500 Internal Server Error — máy chủ gặp sự cố

1.2.3. Website tĩnh và website động; tầng frontend – backend

Dựa trên cơ chế sinh nội dung, website được phân thành hai loại. Bảng so sánh dưới đây làm rõ sự khác biệt và định vị phạm vi của học phần:

Tiêu chí	Website tĩnh	Website động
Cơ chế nội dung	Cố định, viết sẵn trong tệp	Sinh theo dữ liệu và người dùng
Công nghệ	HTML, CSS, JavaScript	Thêm backend + cơ sở dữ liệu
Hiệu năng & chi phí	Cao, chi phí thấp	Linh hoạt hơn, chi phí cao hơn
Phạm vi học phần	Trọng tâm học phần	Ngoài phạm vi

Cần lưu ý rằng ngay cả các ứng dụng web động phức tạp nhất cũng kết xuất giao diện cuối cùng bằng ba ngôn ngữ frontend HTML, CSS và JavaScript. Do đó, việc làm chủ thiết kế web tĩnh phía client là điều kiện tiên quyết cho mọi hướng phát triển web về sau.

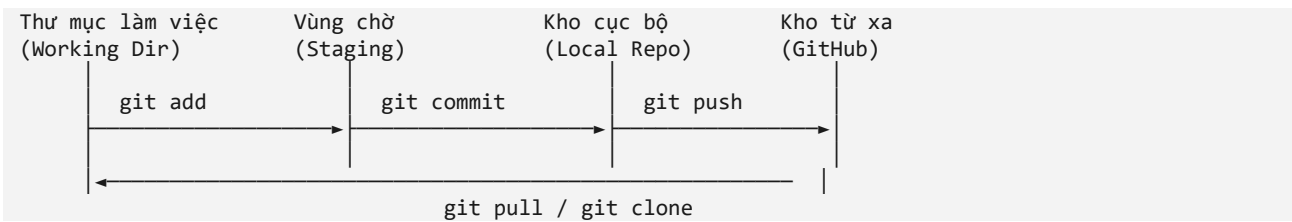
1.2.4. Hệ thống quản lý phiên bản và kiến trúc của Git

Hệ thống quản lý phiên bản (Version Control System – VCS) là công cụ ghi lại lịch sử thay đổi của mã nguồn, cho phép hoàn tác, so sánh và cộng tác nhiều người mà không xung đột. Git là VCS phân tán phổ biến nhất hiện nay. Bảng dưới đây so sánh hai mô hình VCS để làm rõ ưu thế của mô hình phân tán:

Tiêu chí	VCS tập trung	VCS phân tán (Git)
Bản sao kho	Chỉ ở máy chủ trung tâm	Mỗi máy có bản sao đầy đủ
Làm việc offline	Hạn chế	Đầy đủ chức năng

Tiêu chí	VCS tập trung	VCS phân tán (Git)
Rủi ro điểm chết	Cao (mất máy chủ = mất lịch sử)	Thấp (nhiều bản sao)

Trong Git, một thay đổi đi qua ba vùng làm việc trước khi được đồng bộ lên kho từ xa. Kiến trúc này được mô tả như sau:



Giải thích: Lệnh git add đưa thay đổi từ thư mục làm việc vào vùng chờ; git commit ghi thành một mốc trong kho cục bộ; git push đồng bộ các mốc lên kho từ xa GitHub. Chiều ngược lại, git pull (hoặc git clone) mang thay đổi từ kho từ xa về máy.

1.2.5. Xuất bản trang tĩnh với GitHub Pages

GitHub Pages là dịch vụ lưu trữ (hosting) miễn phí cho website tĩnh, xuất bản trực tiếp từ một kho GitHub và tự động cấp chứng chỉ HTTPS. Ưu điểm nổi bật là chi phí bằng không, tích hợp liền mạch với quy trình Git và độ tin cậy cao. Hạn chế là chỉ phục vụ nội dung tĩnh (không thực thi mã phía máy chủ) — song điều này hoàn toàn phù hợp với phạm vi học phần. Ba ràng buộc kỹ thuật cần tuân thủ: (i) sử dụng đường dẫn tương đối cho tài nguyên; (ii) tệp trang chủ phải mang tên index.html và đặt ở thư mục gốc; (iii) chỉ triển khai nội dung tĩnh.

1.3. Ví dụ minh họa thực tế và Mã nguồn

Các ví dụ dưới đây gắn với dự án xuyên suốt của học phần — website giới thiệu "Nhà hàng Món Nghệ" — nhằm bảo đảm tính nhất quán và liên tục qua các chương.

Ví dụ 1.1. Cấu trúc khung của một tài liệu HTML5

```

<!DOCTYPE html>           <!-- Khai báo chuẩn HTML5 -->
<html lang="vi">           <!-- Ngôn ngữ trang: tiếng Việt -->
<head>                     <!-- Vùng thông tin ẩn (metadata) -->
  <meta charset="UTF-8">    <!-- Bảng mã Unicode: hiển thị đúng tiếng Việt -->
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0"> <!-- Hiển thị đúng trên di động -->
  <title>Nhà hàng Món Nghệ</title> <!-- Tên hiển thị trên tab trình duyệt -->
</head>
<body>                     <!-- Vùng nội dung người dùng nhìn thấy -->
  <h1>Nhà hàng Món Nghệ</h1>
</body>
</html>
  
```

Giải thích: Dòng <!DOCTYPE html> bắt buộc phải đứng đầu để trình duyệt áp dụng chuẩn HTML5. Thẻ meta charset="UTF-8" quyết định việc hiển thị đúng dấu tiếng Việt; thiếu nó, trang thường bị lỗi phông. Thẻ meta viewport là điều kiện cần cho thiết kế đáp ứng. Toàn bộ khung này có thể sinh tự động trong VS Code bằng Emmet: gõ dấu ! rồi nhấn phím Tab.

Ví dụ 1.2. Quy trình khởi tạo và đồng bộ kho mã với Git

```

git init                  # Khởi tạo kho Git trong thư mục hiện tại
git add .                 # Đưa toàn bộ thay đổi vào vùng chờ
git commit -m "Khởi tạo trang chủ" # Ghi một mốc kèm thông điệp mô tả
git branch -M main        # Đặt nhánh chính tên là main
git remote add origin <URL-repo> # Liên kết với kho từ xa trên GitHub
git push -u origin main   # Đẩy các mốc lên GitHub
  
```

Giải thích: Chuỗi lệnh này thể hiện đầy đủ một chu trình từ kho cục bộ đến kho từ xa. Tùy chọn -m cung cấp thông điệp commit ngắn gọn, có ý nghĩa — một thực hành chuyên nghiệp giúp lần vết

lịch sử. Tùy chọn -u ở lệnh push thiết lập nhánh theo dõi, nhờ đó các lần đẩy sau chỉ cần gõ git push.

Lưu ý: GitHub không chấp nhận mật khẩu tài khoản khi thao tác qua dòng lệnh. Người dùng phải tạo một Personal Access Token trong phần Settings và dùng token này thay cho mật khẩu.

Ví dụ 1.3. Đường dẫn tương đối và tuyệt đối trên GitHub Pages

```
<!-- ĐÚNG: đường dẫn tương đối, hoạt động trên GitHub Pages -->


<!-- SAI: dấu gạch chéo ở đầu trở về gốc tên miền, gây hỏng ảnh -->

```

Giải thích: Website trên GitHub Pages thường nằm trong một thư mục con của tên miền (ví dụ tenSV.github.io/ten-repo/). Đường dẫn bắt đầu bằng dấu gạch chéo sẽ trở về gốc tên miền, bỏ qua thư mục con, khiến trình duyệt không tìm thấy tài nguyên. Do đó, luôn dùng đường dẫn tương đối cho các tệp nội bộ.

1.4. Bài tập thực hành và Case Study

Bài tập 1.1 (Mức Nhận biết)

Trình bày bằng lời chu trình Yêu cầu – Phản hồi khi một người dùng truy cập địa chỉ <https://monnghe.vn>. Chỉ rõ vai trò của DNS, giao thức HTTP và ý nghĩa của mã trạng thái 200 và 404 trong quá trình đó.

Bài tập 1.2 (Mức Vận dụng)

Tạo một thư mục dự án chứa một tệp index.html tối thiểu, khởi tạo kho Git, thực hiện hai lần commit tương ứng với hai thay đổi khác nhau, sau đó đẩy lên GitHub. Dùng lệnh git log để chụp lại lịch sử hai mốc và giải thích ý nghĩa mỗi mốc.

Case Study (Mức Tổng hợp) — Tình huống doanh nghiệp

Bối cảnh:

Một startup ẩm thực địa phương cần công bố gấp một trang giới thiệu (landing page) cho nhà hàng Món Nghệ trong vòng 48 giờ, với ngân sách hạ tầng bằng không. Nhóm gồm ba thành viên cùng chỉnh sửa mã nguồn và yêu cầu trang phải có địa chỉ công khai, hỗ trợ HTTPS để tạo niềm tin cho khách hàng.

Yêu cầu:

- Đề xuất giải pháp hạ tầng đáp ứng ràng buộc chi phí bằng không và có HTTPS.
- Thiết kế quy trình cộng tác mã nguồn cho ba thành viên, tránh ghi đè lẫn nhau.
- Nêu quy trình từ khi viết mã đến khi trang xuất hiện công khai trên Internet.

Hướng dẫn giải:

1. Chọn GitHub Pages làm nền tảng lưu trữ: miễn phí, có sẵn HTTPS, phù hợp trang tĩnh — thỏa mãn ràng buộc chi phí và bảo mật.
2. Thiết lập một kho GitHub dùng chung; nhóm trưởng mời hai thành viên làm Collaborator. Mỗi thành viên clone kho về máy.
3. Áp dụng kỷ luật cộng tác: mỗi phiên làm việc thực hiện git pull trước khi bắt đầu và git push sau khi hoàn thành để giảm xung đột; đặt tên tệp và thông điệp commit rõ ràng.

4. Quy trình xuất bản: viết index.html ở gốc kho, dùng đường dẫn tương đối cho tài nguyên; commit và push; bật GitHub Pages trong Settings → Pages với nhánh main; sau ít phút, trang có địa chỉ công khai dạng tenSV.github.io/ten-repo với HTTPS.

Giải thích: Case study này rèn năng lực lựa chọn giải pháp có ràng buộc thực tế (chi phí, thời gian, bảo mật, cộng tác) — mô phỏng đúng bài toán mà một kỹ sư frontend thường gặp ở doanh nghiệp nhỏ.

1.5. Câu hỏi ôn tập và Thảo luận

1. Phân tích vì sao tính phi trạng thái (stateless) của HTTP vừa là ưu điểm về khả năng mở rộng, vừa đặt ra thách thức trong việc duy trì phiên làm việc của người dùng. Nêu một cơ chế khắc phục.
2. So sánh mô hình quản lý phiên bản tập trung và phân tán. Trong bối cảnh một nhóm sinh viên làm đồ án, mô hình nào phù hợp hơn và vì sao?
3. Một sinh viên báo rằng ảnh hiển thị bình thường khi mở tệp trên máy nhưng bị mất khi đưa lên GitHub Pages. Hãy chẩn đoán nguyên nhân khả dĩ nhất và đề xuất cách khắc phục.
4. Đánh giá ưu và nhược điểm của việc lựa chọn website tĩnh cho một dự án giới thiệu doanh nghiệp so với website động. Khi nào nên chuyển sang giải pháp động?
5. Thảo luận: vì sao việc ghi thông điệp commit rõ ràng và commit thường xuyên lại được coi là một thực hành chuyên nghiệp quan trọng, đặc biệt khi làm việc nhóm?